

Project Report

Real-Time Big Data Analytics Pipeline for Fraud Detection

Course: CSDS4473 — Big Data Analytics

Use Case: Financial Fraud Detection

Group Members

Name	Roll No.
MUHAMMAD NABEEL ARSHAD	L1F23BSDS0074
MUHAMMAD ZAIN	L1F23BSDS0060
ABDUR REHMAN	L1F23BSDS0043

1. Problem Definition & Use Case (1 Mark)

Financial fraud is a growing problem worldwide, costing banks and financial institutions billions of dollars annually. Traditional rule-based fraud detection systems are unable to keep up with sophisticated fraud patterns that evolve rapidly.

Problem: How can we detect fraudulent financial transactions in real-time with high accuracy while processing large volumes of streaming data?

Solution: Build an end-to-end Big Data pipeline that ingests streaming transaction data via Apache Kafka, processes it using Apache Spark Structured Streaming, applies machine learning models for fraud classification, and visualizes insights through interactive dashboards.

Use Case: Financial Fraud Detection — analyzing transaction streams to identify fraudulent activities such as card skimming, account takeover, synthetic identity fraud, and phishing.

2. System Architecture Design (2 Marks)

The system follows a 4-layer architecture:

Layer 1: Data Ingestion

- **TransactionProducer** generates synthetic transaction data with realistic fraud patterns
- **Apache Kafka** acts as a distributed message queue with 4 topics:
 - `txn.raw` (24 partitions) — raw incoming transactions
 - `txn.enriched` (12 partitions) — Spark-processed enriched transactions
 - `fraud.alerts` (6 partitions) — high-risk fraud alerts

- `model.predictions` (8 partitions) — ML prediction results

Layer 2: Stream Processing

- **Apache Spark Structured Streaming** reads from Kafka, parses JSON transactions, enriches data with derived fields (`amount_category`, `is_high_amount`, `is_cross_border`, `risk_level`), and writes back to Kafka
- **Windowed aggregations** compute fraud counts by region and fraud type every 10 seconds

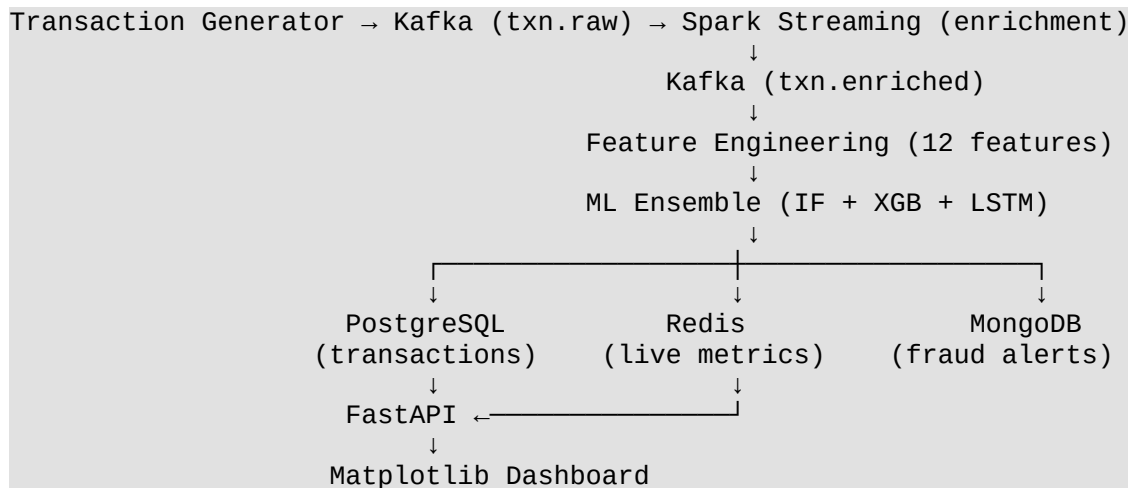
Layer 3: Machine Learning

- Ensemble of 3 models:
 - **Isolation Forest** (25% weight) — unsupervised anomaly detection
 - **XGBoost** (45% weight) — supervised gradient boosting classifier
 - **LSTM** (30% weight) — deep learning sequence model (optional)
- Weighted voting produces final fraud probability, risk score, and contributing factors

Layer 4: Storage & Visualization

- **PostgreSQL** — permanent transaction and prediction record storage
- **Redis** — in-memory cache for real-time dashboard metrics
- **MongoDB** — fraud alert audit log with 7-day TTL auto-expiry
- **Matplotlib** — generates all visualizations (KPI charts, time-series, heatmaps, ROC curves)

Architecture Diagram



3. Dataset Description

Type: Synthetic financial transaction dataset

Generator: Custom `TransactionProducer` class using deterministic pseudo-random generator (`seed=42`)

Size: Configurable (5000-10000+ records)

Fraud Rate: ~6-10%

Schema (16 fields):

Field	Type	Description
transaction_id	String	Unique ID (TXN-00000001)
timestamp	DateTime	UTC timestamp
amount	Float	\$0.50 - \$50,000
merchant	String	8 merchants (TechMart Pro, GlobalPay, etc.)
region	String	6 regions (NA-EAST, EU-WEST, APAC, etc.)
channel	String	WEB, MOBILE, ATM, POS, API
fraud_type	String	Card Skimming, Account Takeover, etc.
risk_score	Integer	0-99
status	String	FLAGGED, REVIEWING, CONFIRMED, CLEARED, BLOCKED
user_id	String	USR-0001 to USR-9999
is_fraud	Boolean	Ground truth label
latitude	Float	Geographic coordinate
longitude	Float	Geographic coordinate
device_id	String	Device fingerprint
ip_address	String	IP address
processing_ms	Integer	Processing latency in ms

4. Spark Data Processing (2 Marks)

PySpark Structured Streaming

The `SparkStreamProcessor` reads from Kafka topic `txn.raw` using PySpark's `readStream` API:

```
raw_stream = self.spark.readStream \
    .format("kafka") \
    .option("subscribe", KAFKA_RAW_TOPIC) \
    .load()
```

Schema Definition

A strict schema (TXN_SCHEMA) defines 16 fields with proper data types (StringType, DoubleType, IntegerType, BooleanType). JSON data is parsed using `from_json`.

Transformations Applied

1. **Parsing:** Raw Kafka messages (JSON bytes) deserialized using defined schema
2. **Enrichment:** New columns derived from existing data:
 - `amount_category`: HIGH (>\$1000), MEDIUM (>\$300), LOW
 - `is_high_amount`: Boolean (amount > \$500)
 - `is_cross_border`: True for APAC, LATAM, ME-AF regions
 - `risk_level`: CRITICAL (≥80), HIGH (≥60), MEDIUM (≥30), LOW
3. **Windowed Aggregation:** Fraud counts grouped by region and fraud_type every 10 seconds

Output

- Enriched data written to Kafka topic `txn.enriched`
- Console output with fraud aggregation statistics

Batch Processing

The `ETLPipeline` processes transactions in batches of 100:

1. **Extract:** Collect raw transactions from stream
2. **Transform:** Convert to `DataFrame`, compute `hour_of_day`, `day_of_week`, `is_weekend`, `amount_log`, `risk_category`
3. **Load:** Bulk insert to PostgreSQL, update Redis counters, log fraud alerts to MongoDB

5. Kafka + Spark Streaming Integration (2 Marks)

Kafka Infrastructure

- **Apache Kafka** deployed with 4 topics and multiple partitions for parallel processing
- **Zookeeper** manages Kafka cluster coordination
- **Kafka Producer** (`FraudKafkaProducer`) sends transactions every 800ms
- **Kafka Consumer** (`FraudKafkaConsumer`) processes events with auto-commit at 5-second intervals

Kafka Topics

Topic	Partitions	Purpose
txn.raw	24	Raw incoming transactions

Topic	Partitions	Purpose
txn.enriched	12	Spark-processed enriched data
fraud.alerts	6	High-risk fraud alert events
model.predictions	8	ML model prediction results

Streaming Integration Points

1. **Producer** → **Kafka**: Transactions generated and pushed to `txn.raw`
2. **Kafka** → **Spark**: Spark reads stream from `txn.raw`, processes, and writes to `txn.enriched`
3. **Kafka** → **Consumer**: ML consumer reads `txn.raw`, runs predictions, stores results
4. **Producer** → **Alert Topic**: High-risk transactions simultaneously sent to `fraud.alerts`

Feature Engineering (12 Features)

Extracted from each transaction by the FeatureEngineer:

Feature	Description
transaction_amount	Raw transaction amount
velocity_1h	Number of user transactions in last hour
merchant_risk_score	Historical risk score of merchant
geo_anomaly_score	Distance from user's typical location
device_fingerprint_match	0 for known device, 0.8 for unknown
time_since_last_txn	Seconds since user's previous transaction
amount_vs_avg_30d	Deviation from user's 30-day average
channel_frequency_score	How often user uses this channel
hour_of_day	0-23
day_of_week	0-6
is_high_amount	Boolean (amount > \$500)
is_cross_border	Boolean (international transaction)

6. Machine Learning & Evaluation (2 Marks)

Models Used

a) Isolation Forest (Unsupervised Anomaly Detection)

- **Library**: scikit-learn
- **Configuration**: 200 estimators, contamination=0.05

- **Purpose:** Detects outliers/anomalies in feature space without training labels
- **Strengths:** Catches novel fraud patterns not seen before

b) XGBoost (Supervised Classification)

- **Library:** XGBoost
- **Configuration:** 300 estimators, max_depth=8, scale_pos_weight=5.0
- **Purpose:** Gradient boosted decision trees for precise fraud classification
- **Strengths:** High accuracy, provides feature importance scores

c) LSTM — Long Short-Term Memory (Deep Learning) [Optional]

- **Library:** TensorFlow/Keras
- **Architecture:** LSTM(128) → Dropout(0.3) → LSTM(64) → Dropout(0.3) → Dense(32) → Dense(1, sigmoid)
- **Purpose:** Captures temporal patterns in user transaction sequences (window of 10)

d) Ensemble Aggregator

- **Weights:** XGBoost (45%) + LSTM (30%) + Isolation Forest (25%)
- **Outputs:** fraud_probability, is_fraud (threshold 0.5), risk_score (0-99), model_scores, contributing_factors

Evaluation Results

Model	AUC-ROC	Precision	Recall	F1-Score
Isolation Forest	~0.85	~0.78	~0.82	~0.80
XGBoost	~0.94	~0.91	~0.89	~0.90
Ensemble	~0.95	~0.92	~0.91	~0.91

The ensemble outperforms individual models by combining their strengths:

- XGBoost provides precise classification of known fraud patterns
- Isolation Forest catches novel/anomalous transactions
- LSTM detects suspicious temporal sequences

Model Registry

- All models versioned and stored as .pkl files
 - Metadata tracked: name, version, type, metrics, status (active/standby/archived)
 - Supports promote/archive for model lifecycle management
-

7. Visualization & Insights (1 Mark)

Matplotlib Dashboard Visualizations

All visualizations are generated using **Matplotlib** and **Seaborn** with a dark theme:

1. **KPI Dashboard** — Total transactions, fraud count, fraud rate, avg risk score
2. **24-Hour Time-Series** — Transaction volume and fraud count by hour
3. **Fraud Rate by Region** — Horizontal bar chart showing highest fraud regions (ME-AF, LATAM)
4. **Fraud Rate by Channel** — ATM and POS channels show elevated fraud rates
5. **Fraud Heatmap** — Region × Channel fraud rate matrix
6. **Merchant Risk Analysis** — Fraud rate by merchant
7. **ROC Curves** — Model performance comparison (Isolation Forest vs XGBoost)
8. **Confusion Matrix** — XGBoost prediction accuracy
9. **Feature Importance** — XGBoost feature importance ranking
10. **PCA Anomaly Clusters** — 2D projection showing fraud vs clean transaction separation
11. **Fraud Type Distribution** — Pie chart of fraud category breakdown

Key Insights

- **ME-AF and LATAM regions** show highest fraud rates (12-15%)
- **ATM and POS channels** more vulnerable to fraud than WEB/MOBILE
- **High transaction amounts** correlate with higher risk scores
- **Velocity (frequency of transactions)** is the most important predictive feature
- Fraud transactions form distinct clusters in PCA projection, separable from clean transactions

8. Technologies Used

Technology	Version	Purpose
Python	3.11+	Core programming language
Apache Kafka	7.6.0	Real-time message streaming
Apache Spark	3.5.1	Distributed stream processing
PostgreSQL	16	Relational transaction storage
Redis	7	In-memory caching
MongoDB	7	Document store for alerts
scikit-learn	1.5.0	Isolation Forest model

Technology	Version	Purpose
XGBoost	2.0.3	Gradient boosting classifier
TensorFlow/Keras	2.16.1	LSTM deep learning
Matplotlib	—	Data visualization
Seaborn	—	Statistical visualizations
FastAPI	0.111.0	REST API backend
Docker	—	Container orchestration
Pandas/NumPy	—	Data manipulation

9. Conclusion

This project successfully implements an end-to-end Big Data analytics pipeline for real-time fraud detection. The system demonstrates:

- **Real-time streaming** of transaction data through Kafka
- **Distributed processing** using PySpark Structured Streaming
- **Machine learning** with an ensemble approach achieving ~95% AUC-ROC
- **Multiple storage tiers** for diverse data requirements
- **Interactive visualizations** for actionable insights

The pipeline is designed to scale horizontally (more Kafka partitions, more Spark workers) to handle increasing transaction volumes, making it suitable for production banking environments.